

Computer Vision Assignment #2

Universität Bern

Contact: Luca.Scharr@unibe.ch

Due date: 17.11.2025

Important: No group work allowed – must be your own coding and writing.

Introduction

In this assignment, you need to upload a zip file to ILIAS which includes:

- (1) A Jupyter Notebook file `Assignment2.ipynb` that contains your Python code.
- (2) A Jupyter Notebook exported to HTML (File / Export Notebook as / HTML).
- (3) A report in PDF format with your answers to the questions in the exercises section.

The zip file name must be `{FirstName}_{LastName}.zip`, where `FirstName` is your first name (*e.g.* Luca), and `LastName` is your last name (*e.g.* Scharr). If your implementation requires auxiliary functions, you must implement those functions inside a corresponding `.py` file. Please state your name at the beginning of the notebook.

Submission guidelines:

In addition to answering the different questions, you are also expected to provide well written submissions. Here are some recommendations to take into consideration.

- Answer questions in the same order as in the assignment and use the same question numbers.
- Ensure the notebook executes sequentially from top to bottom. A TA should be able to reproduce your results by clicking Run All.
- Poorly written submissions may result in point deductions.

Image Inpainting

Inpainting is the problem of filling in regions of an input image g where data is missing. For example, in Figure 1 (left), the texture is masked by black text. Suppose that we are given a function Ω that defines where pixels are missing as follows:

$$\Omega[i, j] = \begin{cases} 0, & \text{if pixel } (i, j) \text{ is missing,} \\ 1, & \text{otherwise.} \end{cases}$$

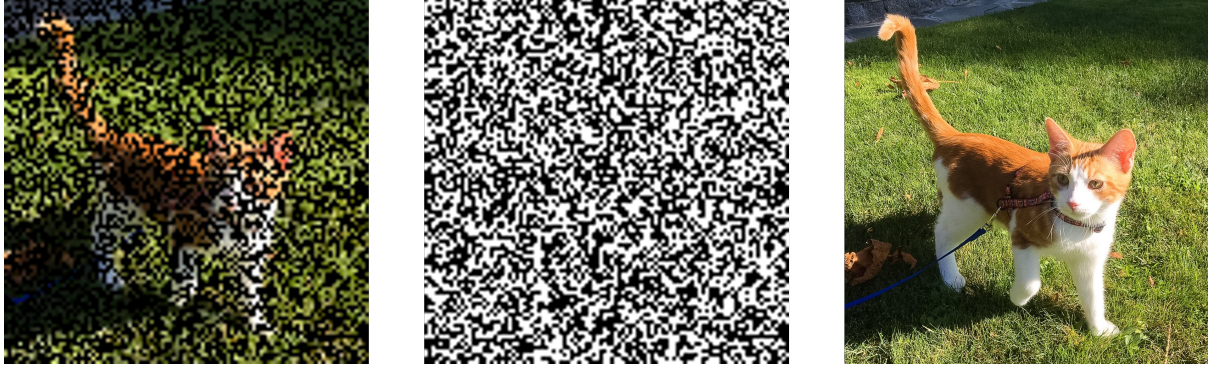


Fig. 1: Visualization of the inpainting problem. (left) input image with blacked out, i.e. masked pixels. (center) mask Ω defining missing pixels. (right) inpainted image.

Formulation

The inpainting problem can be expressed as:

$$\hat{u}_C = \arg \min_{u_C} E[u_C], \quad C \in \{R, G, B\},$$

with

$$E[u_C] = |u_C - g_C|_{\Omega}^2 + \lambda |\nabla u_C|_{2,1}^2.$$

The first term in $E[u_C]$ is the **data term**, encouraging the reconstructed image u_C to match the input image g_C wherever $\Omega = 1$. The second term is the **regularization term** (total variation), and $\lambda > 0$ controls the amount of smoothing. We solve this optimization independently for each color channel $C \in \{R, G, B\}$.

The data term is:

$$|u_C - g_C|_{\Omega}^2 = \sum_{i,j} \Omega[i, j] (u_C[i, j] - g_C[i, j])^2.$$

The total variation term is:

$$|\nabla u_C|_{2,1}^2 = \sum_{i,j} |\nabla u_C[i, j]|_2^2.$$

Hint: Using forward differences,

$$|\nabla u_C[i, j]|_2^2 = (u_C[i + 1, j] - u_C[i, j])^2 + (u_C[i, j + 1] - u_C[i, j])^2.$$

1 Optimization Techniques and Report Questions

1. [10 pts] Finite Difference Approximation of E .

Use forward differences for discretization and show the main calculation steps in your report.

2. [40 pts] Gradient of the Discretized Energy.

Compute the gradient $\nabla_u E$. It is the same for all channels $C \in \{R, G, B\}$. Handle boundary cases carefully and document your derivation.

3. Solvers.

(a) [15 pts] Gradient Descent.

Implement the gradient descent method as:

```
def GD(g, omega, lambda):
    """
    g: color image of size (M, N, 3)
    omega: mask of size (M, N)
    lambda: regularization parameter

    returns: u - inpainted image (M, N, 3)
    """
    return g
```

(b) [25 pts] Linearization + Gauss–Seidel.

Linearize the gradient and use Gauss–Seidel to solve the resulting system iteratively. A numerical Hessian approximation is provided in `hessian_matrix.py`.

```
def LGS(g, omega, lambda):
    """
    g: color image of size (M, N, 3)
    omega: mask of size (M, N)
    lambda: regularization parameter

    returns: u - inpainted image (M, N, 3)
    """
    return g
```

Hint: Use the `scipy.sparse` package functions `triu()`, `tril()`, and `diags()` for efficient splitting.

4. [10 pts] Effect of the Regularization Parameter λ .

Show reconstructed images for very high, very low, and moderate values of λ . Discuss how λ affects the sharpness and smoothness of the reconstruction.

Reference: <https://docs.scipy.org/doc/scipy/reference/sparse.html>